

BÁO CÁO BÀI TIỂU LUẬN LAB 2.1 & LAB 2.2

Môn: Ngôn ngữ lập trình Python

Tên: Phan Trọng Nghĩa

MSSV: 060207013209

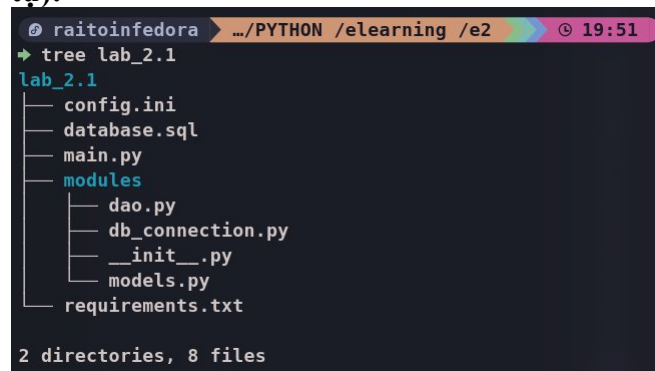
Khoa: Công nghệ thông tin

I. Ý TƯỞNG TỔNG THỂ VÀ MÔ TẢ HỆ THỐNG:

1.1. Kiến trúc:

Trong bài này, em đã dùng mẫu hệ thống thiết kế theo Kiến trúc phân tầng(Layered Architecture) kết hợp với mẫu thiết kế DAO(Data Access Object). Ý tưởng trọng tâm là cô lập hệ thống logic xử lý dữ liệu khỏi giao diện của người dùng, đảm bảo tính dễ bảo trì và di động cao khi chạy trên máy tính khác.

Đây là cây thư mục trực quan của bài 2.1(và bài 2.2 cũng mang kiến trúc tương tự):



```
raitoinfedora .../PYTHON /elearning /e2 19:51
➔ tree lab_2.1
lab_2.1
├── config.ini
├── database.sql
├── main.py
├── modules
│   ├── dao.py
│   ├── db_connection.py
│   ├── __init__.py
│   └── models.py
└── requirements.txt

2 directories, 8 files
```

1.2. Các thành phần hệ thống:

+ **Tầng UI(main.py):** Tiếp nhận input từ bàn phím của người dùng, hiển thị menu dạng console điều hướng và định dạng dữ liệu đầu ra.

+ **Tầng Model Layer(modules/models.py):** ta định nghĩa các lớp đối tượng students và courses để bao đóng dữ liệu theo tư duy OOP.

+ **Tầng Data Access Layer(modules/dao.py):** nơi chứa mã sql, nhận đối tượng từ tầng UI->thực thi truy vấn->trả về kết quả dưới dạng object hoặc list object.

+ **Tầng Kết nối & cấu hình(modules/db_connection.py & config.ini):** Đọc thông số tại thời điểm thực thi, mục đích là để hệ thống tương thích với thông số PostgreSQL của máy tính khác.

1.3. Mô tả Database:

a) Lab 2.1:

Hệ thống quản lý Sinh viên (students) và Khóa học (courses). Mối quan hệ này được chuẩn hóa thông qua bảng trung gian là Đăng ký học phần (enrollment). Bảng enrollment chứa hai khóa ngoại hướng về hai bảng chính và thiết lập ràng buộc UNIQUE (student_id, course_id) để ngăn chặn việc một sinh viên đăng ký trùng một môn học hai lần.

b) Lab 2.2:

Hệ thống quản lý thực thể Sách (books), Thành viên (members) và bảng quan hệ mượn trả (borrow_records). Bảng mượn trả quản lý ngày mượn (borrow_date), hạn trả (due_date) và ngày trả thực tế (return_date) để phục vụ tính năng lọc danh sách quá hạn.

II. MÃ NGUỒN(SOURCE CODE):

Để tối ưu hóa không gian báo cáo và giúp thầy chạy thử, đánh giá thuận tiện hơn, em xin phép được push toàn bộ source code của 2 bài này lên nền tảng Github ạ. Em cảm ơn thầy. Link repo:

https://github.com/letmepassdsa-raito/uth_python_elearning_2

III. GIẢI THÍCH CHỨC NĂNG VÀ HÀM CHÍNH:

BÀI LAB 2.1:

3.1. Cơ chế kết nối động(DatabaseConnection.get_connection)

+ **Chức năng:** Tự động đọc file cấu hình bên ngoài để sinh đối tượng quản lý phiên kết nối (connection)

3.2. Các nhóm hàm CRUD(StudentDAO, CourseDAO)

+ **Hàm thêm: (add) và cập nhật (update):** Nhận tham số đầu vào là một đối tượng (student hoặc course). Sử dụng with connection.cursor() as cursor để tự động giải phóng tài nguyên con trỏ sau khi truy vấn.

+ **Quản lý giao dịch(transaction):** tất cả các hàm ghi dữ liệu đều tích hợp connection.commit() nếu thực thi thành công và connection.rollback() trong khối except để đưa cơ sở dữ liệu về trạng thái an toàn nếu xảy ra xung đột dữ liệu.

+ **Cơ chế tìm kiếm (search):** Sử dụng toán tử ILIKE trong PostgreSQL kết hợp chuỗi định dạng wildcard %s của Python để hỗ trợ tìm kiếm gần đúng, không phân biệt chữ hoa hay chữ thường.

3.3. Nhóm hàm liên kết Nhiều - Nhiều (EnrollmentDAO)

+ **Hàm Đăng ký môn (add):** Nhận cặp khóa student_id và course_id. Khi thực thi, nếu cặp khóa này vi phạm ràng buộc unique_student_course hoặc vi phạm khóa ngoại (SV hoặc Môn không tồn tại) ở tầng DB, khối except sẽ bắt giữ lỗi ngoại lệ và trả về False.

+ **Hàm tra cứu nâng cao (search_by_student & search_by_course):** Thực thi câu lệnh INNER JOIN liên kết 3 bảng dữ liệu (enrollment, students, courses). Hàm ánh xạ dữ liệu kết hợp để hiển thị đầy đủ thông tin chi tiết của sinh viên và tên môn học.

3.4. Vòng lặp điều hướng giao diện (AppUI)

+ **Cơ chế hoạt động:** Sử dụng vòng lặp vô hạn while True kết hợp cấu trúc rẽ nhánh if-elif-else phân cấp để tạo hệ thống Menu.

+ **Xử lý dữ liệu thô:** Áp dụng phương thức .strip() để xóa khoảng trắng thừa ở hai đầu chuỗi nhập vào và .isdigit() để ép kiểu an toàn cho trường dữ liệu số (credits), loại bỏ nguy cơ crash chương trình khi người dùng nhập sai kiểu dữ liệu.

BÀI LAB 2.2:

3.1. Cơ chế ánh xạ đối tượng đa hình (BookDAO.map_row_to_object)

+ **Chức năng:** Chuyển đổi dữ liệu thô (row) truy vấn từ cơ sở dữ liệu thành các đối tượng Python tương ứng (Novel, Textbook, ScienceBook).

+ **Kỹ thuật áp dụng:** Kiểm tra giá trị của cột `book_type`. Dựa vào đó để gọi đúng hàm khởi tạo của lớp con tương ứng, qua đó thể hiện rõ tính đa hình (Polymorphism) trong lập trình OOP.

3.2. Các nhóm hàm CRUD (BookDAO, MemberDAO)

+ **Hàm thêm (add) và cập nhật (update):** Nhận tham số đầu vào là đối tượng Book hoặc Member. Riêng đối với Book, áp dụng hàm `getattr()` của Python để bắt các thuộc tính con (genre, subject, field), nếu không có sẽ tự động gán NULL (phù hợp với thiết kế Single Table Inheritance).

+ **Quản lý giao dịch (transaction):** Sử dụng khối try...except kết hợp `connection.commit()` và `connection.rollback()` để đảm bảo toàn vẹn dữ liệu giống bài 2.1.

3.3. Nghiệp vụ Mượn - Trả sách (BorrowDAO.borrow_book, return_book)

+ **Chức năng mượn sách (borrow_book):** Kiểm tra sách có sẵn không (status = 0). Sau đó thực thi đồng thời 2 lệnh: thêm bản ghi vào `borrow_records` và cập nhật status = 1 (Đã mượn) cho bảng `books`.

+ **Tính toàn vẹn dữ liệu:** Bắt buộc sử dụng Transaction (commit/rollback) tại đây, để nếu lỗi xảy ra ở bảng `books` thì bản ghi ở bảng `borrow_records` cũng bị hủy, tránh hiện tượng lỗi dữ liệu "bóng ma".

3.4. Nghiệp vụ Thống kê quá hạn (BorrowDAO.get_overdue_records)

+ **Chức năng:** Lấy danh sách thông tin người mượn và cuốn sách đã quá hạn trả.

+ **Kỹ thuật truy vấn:** Dùng truy vấn INNER JOIN liên kết 3 bảng. Kết hợp điều kiện chưa trả (`return_date` IS NULL) và hạn trả nhỏ hơn ngày hiện tại (`due_date < CURRENT_DATE`) trực tiếp bằng RDBMS thay vì xử lý bằng vòng lặp Python để tối ưu hiệu năng.

IV. HƯỚNG DẪN CHẠY THỬ CHƯƠNG TRÌNH:

Để chương trình chạy được trên một máy tính bất kỳ thì ta cần tuân theo các bước sau:

Bước 1: Khởi tạo hệ thống

Dùng công cụ pgAdmin hoặc DBeaver. Tạo một cơ sở dữ liệu mới tên là `student_management` đối với bài lab 2.1 hay `library_management` đối với bài lab 2.2. Dán toàn bộ nội dung file `database.sql` vào và nhấn nút run(F5) để khởi tạo cấu trúc bảng.

Bước 2: Cấu hình thông số kết nối

Mở file config.ini nằm trong thư mục gốc của dự án. Thay đổi giá trị tại dòng password = ... thành mật khẩu quản trị PostgreSQL chính xác của máy tính hiện tại đang chạy

Bước 3: Cài đặt thư viện Driver phụ thuộc

Mở terminal hoặc Command Prompt tại thư mục chứa dự án và thực thi lệnh sau để cài đặt thư viện kết nối:

```
pip install -r requirements.txt
```

Bước 4: Khởi chạy ứng dụng

Thực thi tệp tin giao diện chính bằng lệnh:

```
python main.py
```

V. KẾT QUẢ THỰC NGHIỆM (TESTING RESULT):

5.1. Kết quả thực nghiệm LAB 2.1 (Quản lý Sinh viên - Khóa học):

Khởi tạo thành công 2 database cho 2 bài:

List of databases								
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU Locale	ICU Rules	Access privileges
db01	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
db02	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
db04	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
library_management	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
postgres	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
pythonlab2	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
student_management	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
template0	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres +
								postgres=CTc/postgres
template1	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/postgres +
								postgres=CTc/postgres

Chạy chương trình->thêm sinh viên:

```
Nhập lựa chọn của bạn (0-3): 1
--- QUẢN LÝ SINH VIÊN ---
1. Hiện thị danh sách sinh viên
2. Thêm sinh viên mới
3. Sửa thông tin sinh viên
4. Xóa sinh viên
5. Tìm kiếm sinh viên
0. Quay lại menu chính
Nhập lựa chọn (0-5): 2
Nhập MSSV: 060207013209
Nhập họ tên: Phan Trọng Nghĩa
Nhập ngày sinh (YYYY-MM-DD) hoặc bo'trông: 2007-12-02
Nhập email: raitofoacelonmuskvn@gmail.com
Nhập số điện thoại: 0386415434
Nhập địa chỉ: Tỉnh Lâm Đồng
[THÀNH CÔNG]: Đã thêm sinh viên vào hệ thống.
```

Hiện thị danh sách:

```
--- QUẢN LÝ SINH VIÊN ---
1. Hiện thị danh sách sinh viên
2. Thêm sinh viên mới
3. Sửa thông tin sinh viên
4. Xóa sinh viên
5. Tìm kiếm sinh viên
0. Quay lại menu chính
Nhập lựa chọn (0-5): 1
ID: 060207013209 | Name: Phan Trọng Nghĩa | DOB: 2007-12-02 | Email: raitohoacelonmuskvn@gmail.com
```

5.2. Kết quả thực nghiệm LAB 2.2 (Quản lý Thư viện):

Chạy file:

```
➔ python main.py

=====
HỆ THỐNG QUẢN LÝ THƯ VIỆN ĐẠI HỌC
=====
1. Quản lý danh mục Sách
2. Quản lý Thành viên thư viện
3. Nghiệp vụ Mượn sách
4. Nghiệp vụ Tra'sách
5. Thống kê danh sách sách MƯỢN QUÁ HẠN
0. Thoát chương trình
=====
Nhập lựa chọn (0-5):
```

Thêm sách:

```
--- QUẢN LÝ DANH MỤC SÁCH ---
1. Hiện thị tất cả'sách
2. Thêm sách mới
3. Sửa thông tin sách
4. Xóa sách khỏi hệ thống
5. Tìm kiếm sách
0. Quay lại Menu chính
Chọn chức năng (0-5): 2
Mã sách: 001
Tên sách: Machine Learning
Tác giả: Phan Trong Nghia
Số'trang: 500
Năm xuất bản: 2026
Chủng loại: 1. Tiểu thuyết | 2. Sách giáo khoa | 3. Sách khoa học
Chọn (1-3): 3
Lĩnh vực (CNTT, Điện tử,...): CNTT
[THÀNH CÔNG]: Thêm sách hoàn tất.
```

Hiện thị sách:

```
--- QUẢN LÝ DANH MỤC SÁCH ---
1. Hiện thị tất cả'sách
2. Thêm sách mới
3. Sửa thông tin sách
4. Xóa sách khỏi hệ thống
5. Tìm kiếm sách
0. Quay lại Menu chính
Chọn chức năng (0-5): 1
[Science] ID: 001 | Title: Machine Learning | Field: CNTT | Status: Có sẵn
```

